Сопоставление с образцом

Косарев Дмитрий

11 октября 2025 г.

Дата сборки: 10 октября 2025 г.



Сопоставление с образцом

Существенная часть функционального программирования

Два основных подхода:

- *Автоматы с возвратами* минимизируют размер кода (статическая стоимость)
- *Диаграммы решений* минимизируют количество проверок (динамическая стоимость)

Оглавление

1. Обзор

2. Автоматы с возвратами
   - Оптимизации

3. Диаграммы решений

Определение (Соответствие образцу)

Пусть p будет па́ттерном, а v значением какого-то типа. Значение v подходит под образец p (англ. p matches v) (записывается $p \preccurlyeq v$), если подходит одно из правил ниже

$$
\begin{aligned}
_ &\preccurlyeq v \\
x &\preccurlyeq v \\
(p_1 \mid p_2) &\preccurlyeq v && \text{т.т.к. } p_1 \preccurlyeq v \text{ или } p_2 \preccurlyeq v \\
C(p_1; \ldots; p_a) &\preccurlyeq C(v_1; \ldots; v_a) && \text{т.т.к. } (p_1; \ldots; p_a) \preccurlyeq (v_1; \ldots; v_a) \\
(p_1; \ldots; p_a) &\preccurlyeq (v_1; \ldots; v_a) && \text{т.т.к. } p_i \preccurlyeq v_i, \forall i \in [1 \ldots a]
\end{aligned}
$$

Оглавление

1. Обзор

2. Автоматы с возвратами
   - Оптимизации

3. Диаграммы решений

Во что будем компилировать

- `let (x1 e1) (x2 e2) ... (xn en) e`
- Статические исключения и их ловля. `exit n` и `catch e with n`

```
switch l with
  case c1: l1
  ...
  case ck: lk
  default: d
```

- Ветку default можно опускать, тогда будем писать `switch*`
  Потом они компилируются в более простые инструкции
- `field n x`
- Последовательное исполнение `l1; l2` и `()`

Начальное состояние

match x **with** p^1 -> l_1 | p^1 -> l_2 | ... | p^m -> l_m

$$\vec{x} = (x_1 ... x_n), \quad P \to L = \begin{bmatrix} p_1^1 & p_2^1 & \cdots & p_n^1 & \to & l^1 \\ p_1^2 & p_2^2 & \cdots & p_n^2 & \to & l^2 \\ \vdots & \vdots & \ddots & \vdots & & \\ p_1^m & p_2^m & \cdots & p_n^m & \to & l^m \end{bmatrix}$$

catch $C\left(\vec{x}, \begin{bmatrix} p^1 & \to & l^1 \\ p^2 & \to & l^2 \\ \vdots & & \vdots \\ p^m & \to & l^m \end{bmatrix}\right)$ **with** (**failwith** "Partial␣match")

Классическая схема (1/5)

1. Если $\mathfrak{n} = 0$

$$C\Big((), \begin{bmatrix} \to & \mathfrak{l}^1 \\ \to & \mathfrak{l}^2 \\ & \vdots \\ \to & \mathfrak{l}^m \end{bmatrix}\Big) = \mathfrak{l}^1$$

Если $\mathfrak{n} \neq 0$ то будут 4 варианта

2. В первом столбце все переменные
3. В первом столбце все конструкторы
4. Только одна строка с or-па́ттернами
5. Предыдущие случаи не подходят

Классическая схема (2/5). Правило для переменных

Если первый столбец из переменных $(y^1, \dots, y^n)$

$$C(\vec{x}, P \to L) = C((x_2 \dots x_n), P' \to L'),$$

где

$$P' \to L' = \begin{bmatrix} p_2^1 & \cdots & p_n^1 & \to & \text{let } (y^1\ x_1)\ l^1 \\ p_2^2 & \cdots & p_n^2 & \to & \text{let } (y^2\ x_1)\ l^2 \\ & \vdots & & & \\ p_2^m & \cdots & p_n^m & \to & \text{let } (y^m\ x_1)\ l^m \end{bmatrix}$$

Пример:

```
match (x1, x2) with
| (a, p21) -> l1
| (b, p22) -> l2
```

$\Longrightarrow$

```
match x2 with
| p21 -> let a = x1 in l1
| p22 -> let b = x1 in l2
```

Классическая схема (3/5). Правило конструкторов

Все паттерны в 1м столбце — это конструкторы, они образуют множество C. Под каждый конструктор *специализируем* матрицу

Для каждого конструктора $c \in C$ объявляем `r(c)`

p_1^i	$S(c, P \to L)$
$c(q_1^i, \dots, q_a^i)$	$q_1^i, \dots, q_a^i, p_2^i, \dots, p_n^i$
$c'(q_1^i, \dots, q_b^i), c \neq c'$	нет строки

```
r(c) = (let (y1 (field 0 x1))
            ...
            (ya (field (a-1) x1))
       C((y1, ..., ya; x2, ..., xn); S(c; P → L)))
```

Финальная схема компиляции:

```
switch x1 with
case c1 : r1(c1) ... case ck : r(ck)
default: exit
```

Классическая схема (3/5). Правило конструкторов. Пример

```
match (x1, x2) with
| C1 a,      ... -> l1
| C1 b,      ... -> l2
| C2 (a, b), ... -> l3

(* compiles into *)

switch Obj.(tag (repr x1)) of
| C1 -> match Obj.(field 0 (repr x1)), x2 with
        | a, ... -> l1
        | b, ... -> l2
| C2 -> match Obj.(field 0 (repr x1)), Obj.(field 1 (repr x1)), x2 with
        | a,b, ... -> l3
```

Классическая схема (4/5). Or-паттерны

Наша матрица — одна строка, начинающаяся с or-паттернов.

$$P = ((q_1 \mid \cdots \mid q_o)\ p_2 \ldots p_n \to l)$$

$$C\Big((x_1), \begin{bmatrix} q_1 \to & () \\ & \vdots \\ q_o \to & () \end{bmatrix}\Big); \quad C\Big((x_2, \ldots, x_n), (p_2, \ldots, p_n \to l)\Big)$$

- 👍 Мы не дублировали кода
- 👎 Но в or-паттернах не поддерживаются переменные, так как у них область видимости — $()$

Классическая схема (5/5). Предыдущие не подошли

англ. mixture rule

Если ничего из предыдущего не подходит, то матрица $\mathrm{P} \to \mathrm{L}$ разрезается горизонтально на две матрицы $\mathrm{P}_1 \to \mathrm{L}_1$ и $\mathrm{P}_2 \to \mathrm{L}_2$, так что первая матрица — это максимальный префикс исходной, такой, что правила для переменной, конструктора или or-паттернов применимо

$$\texttt{catch}\ \ \mathrm{C}(\vec{\mathrm{x}}; \mathrm{P}_1 \to \mathrm{L}_1)\ \texttt{with}\ \mathrm{C}(\vec{\mathrm{x}}; \mathrm{P}_2 \to \mathrm{L}_2)$$

Оптимизации порождаемых `let` будут в других частях компилятора

Оптимизируем mixture rule. Пример $C((lx\ ly); (P \to L))$

```
let merge lx ly = match lx,ly with
| [], _ -> 1
| _, [] -> 2
| x::xs, y::ys -> 3
```

$$(P \to L) = \begin{bmatrix} [\,] & _ & \to & 1 \\ _ & [\,] & \to & 2 \\ x :: xs & y :: ys & \to & 3 \end{bmatrix}$$

mixture rule применяем дважды

$$(P_1 \to L_1) = ([\,]\ _ \to 1)$$
$$(P_2 \to L_2) = (_\ [\,] \to 2)$$
$$(P_3 \to L_3) = (x :: xs\ y :: ys \to 3)$$

Оптимизируем mixture rule. Пример $C((lx\ ly); (P \to L))$

```
let merge lx ly = match lx,ly with
| [], _ -> 1
| _, [] -> 2
| x::xs, y::ys -> 3
```

$$(P \to L) = \begin{bmatrix} [\,] & _ & \to & 1 \\ _ & [\,] & \to & 2 \\ x :: xs & y :: ys & \to & 3 \end{bmatrix}$$

mixture rule применяем дважды

$$(P_1 \to L_1) = ([\,]\ _ \to 1)$$
$$(P_2 \to L_2) = (_\ [\,] \to 2)$$
$$(P_3 \to L_3) = (x :: xs\ y :: ys \to 3)$$

$$(P \to L) = \begin{bmatrix} [\,] & _ & \to & 1 \\ x :: xs & y :: ys & \to & 3 \\ _ & [\,] & \to & 2 \end{bmatrix}$$

А теперь переставили 2й и 3й паттерн

$$(P_1 \to L_1) = \begin{bmatrix} [\,]\ _ \to 1 \\ x :: xs\ y :: ys \to 3 \end{bmatrix}$$
$$(P_2 \to L_2) = (_\ [\,] \to 2)$$

```
catch
  (catch
    (switch lx with case []: 1
       default: exit)
   with (catch
    (switch ly with case []: 2
       default: exit)
   with (catch
     (switch lx with
        case (::):
          (switch ly with
            case (::) : 3
            default: exit)
        default: exit))))
with (failwith "Partial match")
```

```
catch
  (catch
    (switch* lx with
      case []: 1
      case (::) :
        (switch ly with
          case (::): 3
          default: exit))
  with
    (switch ly with
      case []: 2
      default: exit))
with (failwith "Partial match")
```

Использование информации о полноте

```
catch
  (catch
    (switch* lx with
      case []: 1
      case (::) :
        (switch ly with
          case (::): 3
          default: exit))
  with
    (switch ly with
      case []: 2
      default: exit))
with (failwith "Partial match")
```

Обратим внимание, что нижний switch не попадает в ветку default, и его можно упростить до 2

```
catch
  (switch* lx with
    case []: 1
    case (::) :
      (switch ly with
        case (::): 3
        default: exit))
with 2
```

Более сложный пример

```
type t = Nil | One of int | Cons of int * t
```

$$(\mathrm{P} \to \mathrm{L}) = \begin{bmatrix} \texttt{Nil} & \texttt{_} & \to & 1 \\ \texttt{_} & \texttt{Nil} & \to & 2 \\ \texttt{One x} & \texttt{_} & \to & 3 \\ \texttt{_} & \texttt{One y} & \to & 4 \\ \texttt{Cons(x,xs)} & \texttt{Cons(y,ys)} & \to & 5 \end{bmatrix}$$

$$(\mathrm{P}_1 \to \mathrm{L}_1) = \begin{bmatrix} \texttt{Nil} & \texttt{_} & \to 1 \\ \texttt{Cons(x,xs)} & \texttt{Cons(y,ys)} & \to 5 \end{bmatrix}$$

$$(\mathrm{P}_2 \to \mathrm{L}_2) = \begin{bmatrix} \texttt{_ Nil} \to 2 \end{bmatrix}$$

$$(\mathrm{P}_3 \to \mathrm{L}_3) = \begin{bmatrix} \texttt{One x _} \to 3 \end{bmatrix}$$

$$(\mathrm{P}_4 \to \mathrm{L}_4) = \begin{bmatrix} \texttt{_ One y} \to 4 \end{bmatrix}$$

```
catch                                  1
 (catch                                2
  (catch                               3
   (switch lx with                     4
    case Nil: 1                        5
    case Cons:                         6
     (switch ly with                   7
      case Cons: 5                     8
      default: exit)                   9
                                       10
    default: exit)                     11
   with                                12
    (switch ly with                    13
     case Nil: 2                       14
     default: exit))                   15
 with                                  16
   (switch lx with                     17
    case One: 3                        18
    default: exit))                    19
with 4                                 20
```

```
catch
 (catch
  (catch
   (switch lx with
    case Nil: 1
    case Cons:
     (switch ly with
      case Cons: 5
      case Nil: (exit 2)
      case One: (exit 4)
    default: (exit 2))
   with (2)
    (switch ly with
     case Nil: 2
     default: (exit 3)))
  with (3)
   (switch lx with
    case One: 3
    default: (exit 4)))
with (4) 4
```

Оптимизация exit'ов.

Мотивационный пример.
`(Cons( .. , .. ), One .. )`

путь 7–13–17–20 хотим превратить в
путь 7–20

У нас появляются номера у **`catch`** и у **`exit`**, но это не очень сильное усложнение схемы компиляции

```
1  catch
2   (catch
3    (catch
4     (switch lx with
5      case Nil: 1
6      case Cons:
7       (switch ly with
8        case Cons: 5
9        case Nil: (exit 2)
10       case One: (exit 4)
11     default: (exit 2))
12    with (2)
13     (switch ly with
14      case Nil: 2
15      default: (exit 3)))
16  with (3)
17    (switch lx with
18     case One: 3
19     default: (exit 4)))
20 with (4) 4
```

Более агрессивные оптимизации

В обработчик 3 можем попасть только из 2, где говорят, что `ly <> Nil`

В обработчик 2 можем попасть из двух мест.

```
 9 (lx ly) = (Cons (_, _) Nil)
11 lx = One v
```

Так как в строке 15 выход срабатывает, только когда `ly <> Nil`, придем к выводу, что первый случай не приводит к обработчику 3

Обычно, такое можно провернуть позже, во время dead code elimination

Подробности в статье [1]

К цифрам для `exit` можно добавить ещё и выражения, которые будут соотвествовать паттернам-переменным. В коде они превратятся просто в мутабельную память, куда будут присваивать. Так мы поддержим перемены в or-паттернах

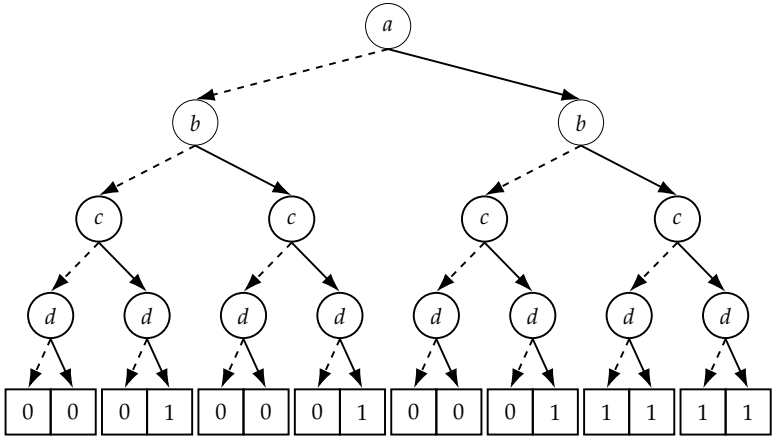
Оглавление

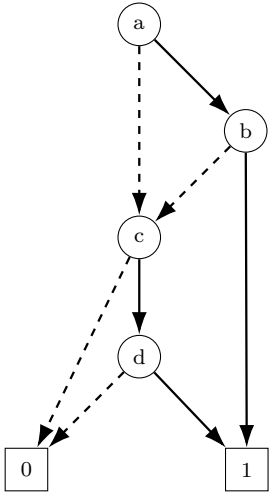
1. Обзор

2. Автоматы с возвратами
   - Оптимизации

3. Диаграммы решений

Диаграмма решений пропозициональной формулы





Decision Trees

Определение (Диаграмма решений)

Это дерево (DAG), где в листьях результаты, в узлах вычисляется некоторые выражения, а на ребрах написаны возможные его значения

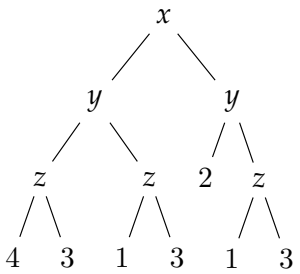
Для нашего случая в листьях результат сопоставления, в узле подвыражение, а на ребрах названия конструкторов

Отличительным свойством является отсутствие повторных проверок

Использование диаграмм решений

```
match x,y,z with
| _,F,T -> 1
| F,T,_ -> 2
| _,_,F -> 3
| _,_,T -> 4
```

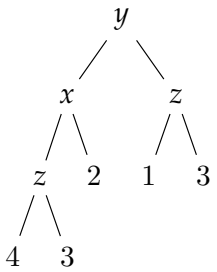
```
if x then
  if y then
    if z then 4 else 3
  else
    if z then 1 else 3
else
  if y then 2
  else
    if z then 1 else 3
```



Использование диаграмм решений

```
match x,y,z with
| _,F,T -> 1
| F,T,_ -> 2
| _,_,F -> 3
| _,_,T -> 4
```

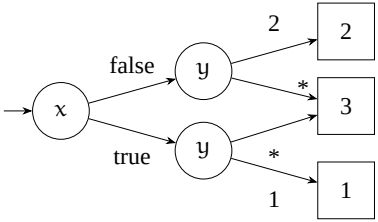
```
if y then
  if x then
    if z then 4 else 3
  else 2
else
  if z then 1 else 3
```

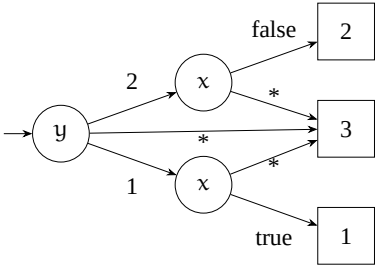


Эвристика necessity [2]

$$P = \begin{bmatrix} \texttt{true} & 1 & \to & \dots \\ \texttt{false} & 2 & \to & \dots \\ _ & _ & \to & \dots \end{bmatrix}$$

$$N = \begin{bmatrix} \bullet & \bullet \\ \bullet & \bullet \\ & \bullet \end{bmatrix}$$





Плохой пример (неэффективный код)

```
type acc =
  | Int of int
  | Clo of int * int
type code =
  | Ldi of int
  | Push
  | Val of acc
  | IOp of int
  | Test of int * int
  | Extend
  | Search of int
  | Pushenv
  | Popenv
  | Mkclos of int
  | Mkclosrec of int
  | Apply
  | Env of int
  | Code of int
```

```
let rec run a s c =
  match (a, s, c) with
  | _, _, Ldi i :: c -> 1
  | _, _, Push :: c -> 2
  | Int n2, Val (Int _)::s, IOp _ :: c -> 3
  | Int 0, _, Test (c2, _) :: c -> 4
  | Int _, _, Test (_, c3) :: c -> 5
  | _, _, Extend :: c -> 6
  | _, _, Search k :: c -> 7
  | _, _, Pushenv :: c -> 8
  | _, Env e :: s, Popenv :: c -> 9
  | _, _, Mkclos cc :: c -> 10
  | _, _, Mkclosrec cc :: c -> 11
  | Clo (cc, ce), Val v :: s, Apply::c -> 12
  | a, Code c :: Env e :: s, [] -> 13
  | a, [], [] -> 14
(* ocamlopt -dlambda bin/main.ml *)
```

Языки с ленивой семантикой (Haskell)

```
or :: Bool -> Bool -> Bool
or True  False = True
or True  True  = True
or False False = False
or False True  = True
```

В *ленивых* языках у нас нет свободы в выборе первого столбца $\Rightarrow$ в *строгих* больше простора для оптимизаций

Например, если делаем ленивую дизъюнкцию, мы просто обязаны мэтчиться слева направо.

Ссылки & Acknowledgements

- Fabrice Le Fessant и Luc Maranget. «Optimizing Pattern Matching». В: *SIGPLAN Not.* 10 (2001).
- Luc Maranget. «Compiling Pattern Matching to Good Decision Trees». В: 2008.
- Kevin D Scott и Norman Ramsey. «When Do Match-compilation Heuristics Matter?» В: 2000.